

Pastael Bag Packing Configurator - Technical Documentation

This document provides a comprehensive, detailed breakdown of the **Pastael Bag Packing Configurator** application. It details the architecture, file responsibilities, calculation formulas, layout adjustment logic (auto & manual), and SVG rendering techniques.

Technology Stack & Libraries

The project is built using modern web technologies to create a high-performance, interactive, and responsive user interface:

1. **React 19 (TypeScript)**: The core UI library. TypeScript ensures type safety across states, components, and mathematical models.
2. **Vite**: The build tool and development server, optimizing code bundling and hot module replacement.
3. **Zustand**: A lightweight, fast, and hook-based state management library used to manage the bag state, pocket instances, and placements.
4. **maxrects-packer (v2.7.3)**: A 2D bin packing library implementing the **MaxRects algorithm**. It is used for the automated packing algorithm.
5. **Lucide React**: Providing modern, clean SVG vector icons.
6. **Vanilla CSS**: Basic CSS styles (`src/index.css`, `src/App.css`) combined with React inline styles for layouts.

File Structure & Directory Walkthrough

Here is the directory map of the codebase and the specific responsibilities of each file:

Path	Category	Purpose & Key Responsibilities
src/main.tsx	Entry Point	Mounts the React application into the DOM (<code>#root div</code>) inside a <code><StrictMode></code> .
src/App.tsx	Entry Point	The main application layout. Arranges the preview panel (left), interactive canvas (center), product list (right), summary (right), and toolbar (bottom) using a flexbox layout.
src/types/Bag.ts	Types	Defines <code>PackingArea</code> (compartments coordinates in cm) and <code>BagTemplate</code> (bag metadata, dimensions, and

Path	Category	Purpose & Key Responsibilities
		compartments).
src/types/Pocket.ts	Types	Defines Pocket (pouch templates with size in cm, image, and rotation permissions).
src/types/Placement.ts	Types	Defines Placement (an instance of a pocket with coordinate positions xCm/yCm, dimensions, rotation state, and whether it was successfully fitted).
src/store/bagStore.ts	State Management	The central store using Zustand. Holds current bag template, selected pouches, positions, canvas zoom, and edit mode (auto/manual). Implements the auto-packer algorithms and placement actions.
src/utils/scaling.ts	Math / Utilities	Defines the scale conversion factor (<code>SCALE = 10</code> , meaning <code>1 cm = 10 px</code>). Provides <code>cmToPx()</code> and <code>pxToCm()</code> .
src/utils/collision.ts	Math / Utilities	Implements 2D AABB (Axis-Aligned Bounding Box) intersection checks (<code>intersects(rectA, rectB)</code>).
src/utils/geometry.ts	Math / Utilities	Implements boundary checks to verify if one rectangle is completely inside another (<code>isInside(inner, outer)</code>).
src/utils/placement.ts	Math / Utilities	Implements a grid search algorithm (<code>findValidPlacement</code>) for manual mode to locate the first available non-overlapping position for a newly added pocket.
src/utils/rotation.ts	Math / Utilities	Defines the rotation degree types (<code>0 90</code>) and toggling logic.
src/components/BagCanvas.tsx	UI Component	Renders the interactive SVG representation of the suitcase, handles mouse drag-and-drop

Path	Category	Purpose & Key Responsibilities
		operations, collisions, double-click rotation, and bounding box colors.
src/components/BagPreview.tsx	UI Component	Computes and displays numerical statistics about bag utilization, total/used/remaining areas in cm ² and px ² for the left and right sides.
src/components/ProductList.tsx	UI Component	Sidebar lists available products to add, shows selected items, displays positions/rotations/compartment offsets, and offers a delete option.
src/components/Summary.tsx	UI Component	Displays simple indicators of total selected items, successfully fitted items, and warnings for items that didn't fit.
src/components/Toolbar.tsx	UI Component	Controls switching between Auto Arrange and Manual Customize modes, and includes a full reset option.

Area Calculations & Conversions

The app maintains all coordinates and sizes in physical metric units (**Centimeters, cm**) for algorithmic precision, and maps them to **Pixels (px)** in the SVG viewer layer.

1. The Scaling Factor

$\text{Scale Factor} = 10 \text{ (1 cm} = 10 \text{ px)}$

Thus:

- **Length Conversion:** $\text{pixels} = \text{centimeters} \times 10$
- **Area Conversion:** Since area scales quadratically: $\text{pixels}^2 = \text{centimeters}^2 \times 100$ (Example: A $30\text{cm} \times 42\text{cm}$ compartment has an area of 1260 cm^2 . In pixels, its area is $300 \text{ px} \times 420 \text{ px} = 126,000 \text{ px}^2$, which is 1260×100).

2. Bag Compartments Layout

The default Suitcase layout has two usable packing areas:

- **Suitcase Size:** $71.8 \text{ cm} \times 46.9 \text{ cm}$ (translated to SVG container as $718 \text{ px} \times 469 \text{ px}$).

- **Left Packing Compartment:**
 - Position: $x = 3.5\text{ cm}$, $y = 2.5\text{ cm}$
 - Size: $\text{Width} = 30.0\text{ cm}$, $\text{Height} = 42.0\text{ cm}$
 - Total Area: 1260 cm^2 ($126,000\text{ px}^2$)
- **Right Packing Compartment:**
 - Position: $x = 38.3\text{ cm}$, $y = 2.5\text{ cm}$
 - Size: $\text{Width} = 30.0\text{ cm}$, $\text{Height} = 42.0\text{ cm}$
 - Total Area: 1260 cm^2 ($126,000\text{ px}^2$)
- **Total Usable Area:** $\text{Total Usable Area} = 1260 + 1260 = 2520\text{ cm}^2$ ($252,000\text{ px}^2$) (Note: The empty area between $x = 33.5\text{ cm}$ and $x = 38.3\text{ cm}$ is the suitcase middle divider/spine and is excluded from packing calculations).

3. Utilization & Remaining Space Computations

In [src/components/BagPreview.tsx](#):

- Fitted items are categorized into left or right using a split threshold of $x = 35.0\text{ cm}$:
 - **Left Side Items:** $\text{placement.xCm} < 35.0$
 - **Right Side Items:** $\text{placement.xCm} \geq 35.0$
- **Used Area Formula** (per compartment): $\text{Used Area} = \sum (\text{placement.widthCm} \times \text{placement.heightCm})$
- **Remaining Area Formula:** $\text{Remaining Area} = \max(0, \text{Total Compartment Area} - \text{Used Area})$
- **Combined Utilization Percentage:** $\text{Utilization \%} = \min\left(100, \text{Round}\left(\frac{\text{Left Used Area} + \text{Right Used Area}}{\text{Total Usable Area}} \times 100\right)\right)$



Packaging Adjustment & Placement Logic

The configurator supports two editing modes: **Auto Arrange** and **Manual Edit**.

```
graph TD
  A[Add Pouch / Change Mode] --> B{Mode?}
  B -- Auto Arrange --> C[Sort Pouches by Area Descending]
  C --> D[Initialize Left MaxRectsPacker]
  D --> E[Try placing in Left Compartment]
  E -- Fits --> F[Assign Left Coordinates]
  E -- Doesn't Fit --> G[Initialize Right MaxRectsPacker]
  G --> H[Try placing in Right Compartment]
  H -- Fits --> I[Assign Right Coordinates]
  H -- Doesn't Fit --> J[Mark as Unfitted]
  F & I & J --> K{All items fitted?}
  K -- No, & first run was without rotation --> L[Re-run packing with rotation enabled]
  L --> M{Does rotation fit more items?}
```

```

M -- Yes --> N[Adopt rotated layout]
M -- No --> O[Adopt non-rotated layout]
K -- Yes / Rotation already tried --> P[Update placements in Zustand Store]

B -- Manual Edit --> Q[Grid Search: scanning compartments top-left to
bottom-right]
Q --> R{Fits without collision?}
R -- Yes --> S[Place at first valid spot]
R -- No --> T[Place at origin, mark as Unfitted]

```

1. Auto Arrange (Bin Packing)

Located in `runAutoArrange` in [src/store/bagStore.ts](#):

1. **Decreasing Sorting:** Sorts all selected pocket instances by surface area in descending order: $\text{Area} = \text{widthCm} \times \text{heightCm}$ *Heuristic: Placing larger objects first avoids leaving unfillable holes that small items would otherwise occupy.*
2. **Sequential Bin Packing:**
 - For each item, it constructs a fresh `MaxRectsPacker` for the Left compartment. It re-inserts all previously successfully placed left items first, then attempts to add the new item.
 - If it fits inside the Left compartment, coordinates are assigned relative to `leftBin.x` and `leftBin.y`.
 - If it does not fit, it repeats the process for the Right compartment.
 - If it fits neither, the item is marked `fitted: false` and coordinates set to $(0,0)$.
3. **Rotation Optimization:**
 - The engine first runs with rotation disabled.
 - If any item fails to fit, it runs the engine again with rotation enabled.
 - It counts the number of fitted items. If enabling rotation increases the amount of fitted pouches, the rotated layout is selected. Otherwise, the non-rotated layout is retained.

2. Manual Customization

- **First-Place Insertion:** When adding a pocket in manual mode, [findValidPlacement](#) scans the compartments in a grid from top-left to bottom-right in $0.5 \times \text{cm}$ increments. It checks:
 1. Is the proposed rectangle fully contained inside the compartment? (`isInside`)
 2. Does it intersect any already placed items? (`intersects`) *It returns the first coordinate that satisfies both, or flags the item as unfitted if none exists.*
- **Interactive Dragging:**
 1. On mouse drag, the offset from the click point to the top-left of the pouch is calculated.

2. As the mouse moves, the new coordinates are updated in real-time.
3. The app dynamically checks if the current mouse coordinate causes an overlap or boundary collision. If invalid, the bounding outline turns red, indicating that releasing the mouse will revert the pouch back to its original position.
4. On mouse release, if the position is valid, the store updates the coordinate; otherwise, it reverts.

- **Manual Rotation:**

1. Double-clicking an item in manual mode swaps its width and height: $\text{newWidthCm} = \text{heightCm}$, $\text{newHeightCm} = \text{widthCm}$
2. The app checks if the swapped rectangle causes any collision or goes out-of-bounds at the current location.
3. If clear, the rotation state changes (0° to 90° or 90° to 0°).

Interactive SVG Rendering & Coordinate Mapping

The canvas is rendered entirely using SVG tags. Since screen coordinates (`clientX`, `clientY`) do not line up with the local SVG coordinate space, mapping is applied.

1. ViewPort Mapping

The SVG is defined with a view box:

```
<svg
  ref={svgRef}
  width={svgWidth}           // cm * 10
  height={svgHeight}         // cm * 10
  viewBox={`0 0 ${svgWidth} ${svgHeight}`}
  ...
/>
```

This ensures the coordinate scale within the SVG code is exactly 1-to-1 with pixels, regardless of the SVG's actual size in the responsive browser layout.

2. Converting Mouse coordinates to SVG coordinates

The [getSvgCoords](#) function maps screen coordinates:

```
const getSvgCoords = (clientX: number, clientY: number) => {
  if (!svgRef.current) return { x: 0, y: 0 };
  const rect = svgRef.current.getBoundingClientRect();
  const svgX = ((clientX - rect.left) / rect.width) * svgWidth;
  const svgY = ((clientY - rect.top) / rect.height) * svgHeight;
  return { x: svgX, y: svgY };
};
```

- `rect.left/rect.top` are screen coordinates of the SVG border.
- `clientX - rect.left` gets the mouse offset from the left of the canvas.
- Dividing by `rect.width` maps this offset to a percentage (0.0 to 1.0).

- Multiplying by `svgWidth` converts the percentage to SVG pixels.

3. Math for Center-Based SVG Rotations

When rendering a rotated pouch, a simple naive rotation shifts the object's origin. To keep the pouch aligned, the application rotates the product image around its **center point** while maintaining a regular rectangle boundary box:

```
const isRotated = pl.rotation !== 0;

// Center of the bounding rectangle box
const cx = x + width / 2;
const cy = y + height / 2;

// Unrotated dimensions of the pouch template
const imgW = cmToPx(product.widthCm);
const imgH = cmToPx(product.heightCm);

// Top-left coordinate to center the unrotated image at (cx, cy)
const imgX = cx - imgW / 2;
const imgY = cy - imgH / 2;

// SVG rotation instruction: rotate(angle, center_x, center_y)
const imageTransform = isRotated ? `rotate(${pl.rotation}, ${cx}, ${cy})` :
undefined;
```

Explanation of Rotation Coordinates:

1. width and height are the *placed/rotated* dimensions. If rotated 90° , width is the product's height and height is the product's width.
2. `cx`, `cy` represents the exact center of this bounding box.
3. We render the image in its original unrotated aspect ratio (`imgW` and `imgH`), centering it on `cx`, `cy`.
4. We apply `transform="rotate(90, cx, cy)"`. This rotates the SVG `<image>` tag around `cx`, `cy`.
5. The rotated image fits the bounding outline box perfectly.



Summary of Collision and Boundary Formulas

Collision Check (`intersects`):

Checks if two axis-aligned rectangles overlap:

```
rectA.x < rectB.x + rectB.width &&
rectA.x + rectA.width > rectB.x &&
rectA.y < rectB.y + rectB.height &&
rectA.y + rectA.height > rectB.y
```

Boundary Containment (`isInside`):

Checks if the inner rectangle is fully inside the container boundaries:

```
inner.x >= outer.x &&
inner.y >= outer.y &&
```

```
inner.x + inner.width <= outer.x + outer.width &&  
inner.y + inner.height <= outer.y + outer.height
```